Own Work Declaration

I/We hereby understand my/our work would be checked for plagiarism or other misconduct, and the softcopy would be saved for future comparison(s).

I/We hereby confirm that all the references or sources of citations have been correctly listed or presented, and I/we clearly understand the serious consequences caused by any intentional or unintentional misconduct.

This work is not made on any work of other students (past or present), and it has not been submitted to any other courses or institutions before.

Signature:

Contributor 1: Amine Benarroudj

Contributor 2: Addison Dobbs

Date:

Sunday, April 12, 2026

CSCE 331 – Programming Language Concepts

Spring 2026

Mini Compiler Using Flex and Bison

Amine Benarroudj and Addison Dobbs

April 12, 2026

Table of Contents

1. Introduction **4**

2. Installation and Setup **4**

2.1 MacOS Set Up 4

2.2 Ubuntu Set Up 5

3. Features **6**

3.1 Arithmetic Operations 6

3.2 Mathematical Functions 6

3.3 Variables and Assignment 6

3.4 Control Flow 7

if / else *7*

while loop *7*

for loop *7*

switch / case / default *7*

4. Implementation **7**

4.1 Lexical Analyzer (lexer.l) 7

4.2 Parser (parser.y) 11

5. Integration and Compilation **17**

Step 1: *17*

Step 2: *17*

Step 3: *17*

Step 4: *18*

6.1 Arithmetic 18

Addition *18*

Subtraction *19*

Multiplication *19*

Division *19*

6.2 Trigonometric and Math Functions 19

Sin *19*

Cos *19*

Square Root *19*

6.3 Variables 20

6.4 Conditional Statements 20

if / else *20*

while *20*

for *20*

switch *20*

7. Analysis **21**

7.1 Limitations 21

7.2 Possible Improvements 21

8. Conclusion **22**

9. References **22**

1. Introduction

In this project, we built a mini compiler using two tools called Flex and Bison. Flex handles lexical analysis, which means it reads the raw text you type and breaks it down into meaningful pieces called tokens, things like numbers, variable names, operators, and keywords. Bison then takes those tokens and applies grammar rules to figure out what the input means and evaluate it, similar to how a real compiler processes source code.

The compiler we built supports a wide range of features. It can handle all basic arithmetic operations like addition, subtraction, multiplication, division, modulo, and exponentiation, and it correctly respects the standard order of operations. It also supports mathematical functions including sine, cosine, tangent, natural log, exponential, square root, and absolute value. On top of that, we added support for named variables, so you can assign a value like x = 10 and reuse it in later expressions.

For control flow, the compiler recognizes if and if/else statements, while loops, for loops, and switch/case blocks with a default clause. We also implemented proper error handling for things like division by zero, taking the log of a negative number, or referencing an undefined variable. The project consists of four files. The lexer is defined in lexer.l, the grammar and evaluation logic live in parser.y, the entry point is main.c, and a ReadMe ties everything together so the whole project can be compiled with a single make command. Overall, this project gave us hands-on experience with every stage of compiler front-end design, from tokenization and grammar definition all the way through expression evaluation and control flow execution.

2. Installation and Setup

The following section will go over how to install Flex and Bison on MacOS and an Ubuntu Linux system.

2.1 MacOS Set Up

The recommended way to install Flex and Bison on macOS is through **Homebrew**. Homebrew is a popular package manager for macOS.

Step 1:
If you do not already have Homebrew installed, you can install it by opening a Terminal window and running the following command:

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow any on-screen prompts to complete the setup.
You can also follow the instruction directly from the website: https://brew.sh

Step 2:

Now that we have Homebrew installed, it's time to install Flex and Bison.

Run:

```
MacBook-Pro ~ % brew install flex bison gcc
```

Step 3:
To ensure your terminal uses the Homebrew versions rather than the outdated system ones, you need to manually add them to your PATH.
Run:

```
MacBook-Pro ~ % echo 'export PATH="/opt/homebrew/opt/bison/bin:$PATH"' >> ~/.zshrc
```

```
MacBook-Pro ~ % source ~/.zshrc
```

Step 4:
Check what version is installed to ensure that everything is indeed installed and up to date.

Run:

```
flex --version && bison --version
```

```
flex 2.6.4 Apple(flex-35)
bison (GNU Bison) 3.8.2
Written by Robert Corbett and Richard Stallman.

Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
(base) absfolder@Amines-MacBook-Pro ~ %
```

As of April, 2026 - the latest versions are flex 2.6.4 and bison 3.8.2

2.2 Ubuntu Set Up

Step 1:
Before doing any, first open a Terminal and make sure your system's package list is up to date.
Run command: **sudo apt update**

Step 2:
Install both tools
Run command: **sudo apt install flex bison**

Step 3:
Check what version is installed to ensure that everything is indeed installed and up to date.
Run command: **flex –version && bison --version**

3. Features

3.1 Arithmetic Operations

Operation	Syntax	Example
Addition	a + b	5 + 3
Subtraction	a - b	5 - 3
Multiplication	a * b	5 * 3
Division	a / b	15 / 3
Modulo	a % b	10 % 3
Exponentiation	a ^ b	2 ^ 3
Unary Negation	-a	-5 + 2

3.2 Mathematical Functions

Function	Syntax	Example
Sine	sin(x)	sin(0) = 0
Cosine	cos(x)	cos(0) = 1
Tangent	tan(x)	tan(0.7854) = 1
Natural Log	log(x)	log(2.71828) ≈ 1
Exponential	exp(x)	exp(1) ≈ 2.71828
Square Root	sqrt(x)	sqrt(25) = 5
Absolute Value	abs(x)	abs(-5) = 5

3.3 Variables and Assignment

Named variables can be assigned and reused across statements. The variable table stores up to 64 variables. Variables are assigned using = and printed using the print keyword. Must use ; at the end of each line you want to execute.

```
x = 10;
y = 3;
x + y;      // equals 13
x = x + 1;
print x;      // equals 11
```

3.4 Control Flow

if / else

The if/else construct evaluates a boolean condition and executes only the matching branch. A global suppress counter ensures the non-taken branch produces no output.

```
if (5 > 3) { print 1; }          // will print 1
if (1 > 5) { print 99; } else { print 0; }          // will print 0
```

while loop

The while construct checks a condition before executing its body block. Due to the single-pass nature of LALR(1) parsing without an AST, the body is evaluated once during reduction. This is a known limitation documented in Section 7.1.

```
while (1 > 0) { print 7; }    // will print [while] condition true; body evaluated
while (0 > 5) { print 9; }    // will print [while] condition false; body skipped
```

for loop

The for construct fully recognizes the three-part header syntax (init; condition; update). As with while, iterative execution requires an AST; the body is evaluated once per reduction pass.

```
for (i = 1; i < 5; i = i + 1) { print i; }          // will print [for] condition true; body evaluated (result: 1)
```

switch / case / default

The switch construct parses an expression and a list of case and default clauses. The grammar recognizes the full switch syntax including fall through-style case lists.

```
switch (2) {
   case 1: print 100;
   case 2: print 200;
   default: print 0;
}                                  // will print 100  200  0  [switch] on value: 2
```

4. Implementation

The compiler is composed of two generated components — a lexical analyzer and a parser — plus a hand-written entry point. Flex and Bison each read a specification file written by the developer and automatically generate the corresponding C source code, which is then compiled together with main.c into a single executable.

4.1 Lexical Analyzer (lexer.l)

The lexical analyzer (lexer) is the first stage of the compiler pipeline.

It reads the raw input stream one character at a time and groups characters into meaningful units called tokens.

The lexer does not assign meaning to these tokens, it simply classifies them and passes them to the parser.

The lexer is specified in lexer.l using Flex regular expression rules.

The prologue section is enclosed in %{ ... %} and contains raw C code that is copied verbatim into the generated lex.yy.c file.

It includes parser.tab.h, which is generated by Bison and contains the numeric constants for every token name (e.g. PLUS = 258, NUMBER = 259). Without this header the lexer would not know what integer value to return for each token. stdlib.h and string.h are included for atof(), strdup(), and strcmp().

lexer.l

```
%{
#include "parser.tab.h"
#include <stdlib.h>
#include <string.h>
%}
```

Token Rule: NUMBER
The token rule pattern matches any integer or floating-point literal in the input. The + quantifier requires at least one digit before the decimal point; the (\.[0-9]+)? part optionally matches a dot followed by one or more digits, enabling decimals like 3.14.

When this pattern matches, the action yylval.dval = atof(yytext);

converts the matched text to a C double using the standard library function atof().

The value is stored in yylval.dval , so the parser can retrieve the numeric value when it processes this token.

The function then returns the token constant NUMBER to the parser.

```
%%

[0-9]+(\.[0-9]+)?       { yylval.dval = atof(yytext); return NUMBER; }
```

Token Rule: IDENT and Keywords
The pattern [a-zA-Z_][a-zA-Z0-9_]* matches any identifier: a word that starts with a letter or underscore and is followed by zero or more letters, digits, or underscores. This single pattern handles both reserved keywords (if, while, sin, etc.) and user-defined variable names (x, myVar, etc.).

Because a keyword like if and a variable name like items both match the same pattern, the action uses a series of strcmp() calls to check whether the matched text is a known keyword.

If it is, the corresponding token constant (IF, WHILE, SIN, etc.) is returned directly.

If no keyword matches, the text is a variable name: it is duplicated into heap memory using strdup() and stored in yylval.sval, then the IDENT token is returned.

Using strdup() is necessary because yytext is overwritten with the next token on the next call to yylex().

```
[a-zA-Z_][a-zA-Z0-9_]* {
    /* Keywords first, then variable names */
    if      (strcmp(yytext, "sin")     == 0) return SIN;
    else if (strcmp(yytext, "cos")     == 0) return COS;
    else if (strcmp(yytext, "tan")     == 0) return TAN;
    else if (strcmp(yytext, "log")     == 0) return LOG;
    else if (strcmp(yytext, "exp")     == 0) return EXP;
    else if (strcmp(yytext, "sqrt")    == 0) return SQRT;
    else if (strcmp(yytext, "abs")     == 0) return ABS;
    else if (strcmp(yytext, "if")      == 0) return IF;
    else if (strcmp(yytext, "else")    == 0) return ELSE;
    else if (strcmp(yytext, "while")   == 0) return WHILE;
    else if (strcmp(yytext, "for")     == 0) return FOR;
    else if (strcmp(yytext, "switch")  == 0) return SWITCH;
    else if (strcmp(yytext, "case")    == 0) return CASE;
    else if (strcmp(yytext, "default") == 0) return DEFAULT;
    else if (strcmp(yytext, "break")   == 0) return BREAK;
    else if (strcmp(yytext, "print")   == 0) return PRINT;
    else {
        /* Variable name: copy into yylval */
        yylval.sval = strdup(yytext);
        return IDENT;
    }
}
```

Token Rules: Operators and Delimiters
Each operator and delimiter is matched by a quoted string pattern.

Flex tries rules in the order they are written, so multi-character operators like == and >= are listed before their single-character prefixes = and >. This ensures the longer match wins.

Each rule simply returns the corresponding token constant to the parser; no value needs to be stored in yylval for operators because the token type itself carries all the information the parser needs.

The full set of operators recognized:

- **Arithmetic** (addition, subtraction, multiplication, division, modulo, exponentiation)
- **Comparison** (equal, not-equal, greater, greater-or-equal, less, less-or-equal)
- **Logical** (AND, OR, NOT)
- **Assignment** (variable assignment)
- **Delimiters** (grouping, blocks, statement terminator, case separator)

```
"=="                    { return EQ; }
"!="                    { return NEQ; }
">="                    { return GTE; }
"<="                    { return LTE; }
">"                     { return GT; }
"<"                     { return LT; }
"&&"                    { return AND; }
"||"                    { return OR; }
"!"                     { return NOT; }

"^"                     { return POW; }
"+"                     { return PLUS; }
"-"                     { return MINUS; }
"*"                     { return MUL; }
"/"                     { return DIV; }
"%"                     { return MOD; }
"="                     { return ASSIGN; }
":"                     { return COLON; }

"("                     { return LPAREN; }
")"                     { return RPAREN; }
"{"                     { return LBRACE; }
"}"                     { return RBRACE; }
";"                     { return SEMI; }
```

Token Rule: Whitespace
The pattern [\t\n\r] matches any space, tab, newline, or carriage-return character.

The action is a bare semicolon ; – which in Flex means do nothing and continue scanning.

This causes all whitespace to be silently discarded.

The lexer never returns a token for whitespace, so the parser never sees it and does not need to handle it in the grammar.

```
[ \t\n\r]               ;   /* skip whitespace */

.                       { fprintf(stderr, "Unknown character: %s\n", yytext); }

%%
```

End-of-Input Function: yywrap()
Flex requires a function called yywrap() to be defined.

It is called automatically when the lexer reaches the end of input.

Returning 1 tells Flex that there is no additional input to process and that scanning should stop. Without this function the program will not compile.

```
int yywrap() { return 1; }
```

4.2 Parser (parser.y)

The parser is the second and main stage of the compiler. It receives a stream of tokens from the lexer and matches them against a context-free grammar written in Bison's BNF-style notation. When a grammar rule is matched ("reduced" in LALR(1) terminology), the embedded C action is executed immediately, computing and propagating values up the parse tree. The parser is specified in parser.y.

Prologue: Includes and Global State

The prologue includes stdio.h for output, math.h for trigonometric and exponential functions, string.h and stdlib.h for string handling and memory allocation. It also declares the prototypes for yyerror() and yylex() so the compiler knows about them.

parser.y

```
%{
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <stdlib.h>
```

Two pieces of global state are defined here. First, a suppress integer counter controls output: whenever its value is greater than zero, all statement actions skip printing.

This is the mechanism that makes if/else work correctly. The branch not taken has its output suppressed by incrementing this counter before its body is reduced, then decrementing it after. Second, the variable table. It's declared here along with the get_var() and set_var() helper functions.

```
double get_var(const char *name) {
    int i;
    for (i = 0; i < var_count; i++)
        if (strcmp(var_table[i].name, name) == 0)
            return var_table[i].value;
    fprintf(stderr, "Warning: undefined variable '%s', using 0\n", name);
    return 0.0;
}

double set_var(const char *name, double val) {
    int i;
    for (i = 0; i < var_count; i++) {
        if (strcmp(var_table[i].name, name) == 0) {
            var_table[i].value = val;
            return val;
        }
    }
    if (var_count < VAR_TABLE_SIZE) {
        strncpy(var_table[var_count].name, name, 63);
        var_table[var_count].value = val;
        var_count++;
    } else {
        fprintf(stderr, "Error: variable table full\n");
    }
    return val;
}
```

The %union Directive
%union directive defines a C union that is used as the type of yylval, the variable through which the lexer passes semantic values to the parser. A union is used instead of a struct because only one field is active at a time: a token is either a number or a name, never both simultaneously.

The two fields are:

- **dval** - a C double, used for NUMBER tokens and for the result of any expr rule
- **sval** - a char pointer, used for IDENT tokens (variable names) allocated with strdup()

Every token and non-terminal in the grammar is then tagged with one of these types using %token <dval>, %token <sval>, or %type <dval>, so Bison knows which union field to use when it accesses $1, $2, etc. inside rule actions.

```
%union {
    double  dval;
    char   *sval;
}
```

Operator Precedence and Associativity
Without explicit precedence declarations, an LALR(1) parser cannot resolve ambiguous grammar rules; for example, whether 2 + 3 * 4 means (2 + 3) * 4 = 20 or 2 + (3 * 4) = 14.

Bison resolves these ambiguities using %left, %right, and %nonassoc declarations listed from lowest to highest precedence.

The precedence table used in this compiler, from lowest to highest priority:

- **%right ASSIGN** - assignment (=) is right-associative: a = b = 5 assigns 5 to b then to a
- **%left OR / AND** - logical operators, lower priority than comparisons
- **%right NOT** - logical NOT is a unary right-associative prefix operator
- **%left EQ NEQ** - equality comparisons
- **%left GT GTE LT LTE** - relational comparisons
- **%left PLUS MINUS** - addition and subtraction
- **%left MUL DIV MOD** - multiplication, division, modulo; higher than add/sub
- **%right UMINUS** - unary minus (e.g. -5); higher than binary arithmetic
- **%right POW** - exponentiation; highest, right-associative so 2^3^2 = 2^(3^2) = 512

The %prec UMINUS directive in the unary minus rule overrides the default precedence of the MINUS token, giving -expr higher priority than binary subtraction.

```
%token <dval> NUMBER
%token <sval> IDENT
%token SIN COS TAN LOG EXP SQRT ABS
%token IF ELSE WHILE FOR SWITCH CASE DEFAULT BREAK PRINT
%token PLUS MINUS MUL DIV MOD POW
%token EQ NEQ GT GTE LT LTE AND OR NOT
%token ASSIGN COLON
%token LPAREN RPAREN LBRACE RBRACE SEMI

%type <dval> expr stmt stmts if_cond

/* Operator precedence (lowest to highest) */
%right ASSIGN
%left  OR
%left  AND
%right NOT
%left  EQ NEQ
%left  GT GTE LT LTE
%left  PLUS MINUS
%left  MUL DIV MOD
%right UMINUS
%right POW

/* Standard dangling-else resolution: prefer shifting ELSE */
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

Grammar Rules: Statements

The stmt non-terminal defines all executable statement types the compiler recognizes. Each rule ends with a C action enclosed in { } that performs the actual computation or output. Every action checks the suppresscounter before printing.

- **expr SEMI** - evaluates an expression and prints its result with printf("= %.6g\n", $1)
- **IDENT ASSIGN expr SEMI** - calls set_var() to store the value, then prints name = value
- **PRINT expr SEMI** - prints the expression value without the = prefix

```
stmt:
    expr SEMI {
        if (!suppress) printf("= %.6g\n", $1);
        $$ = $1;
    }
  | IDENT ASSIGN expr SEMI {
        set_var($1, $3);
        if (!suppress) printf("%s = %.6g\n", $1, $3);
        free($1);
        $$ = $3;
    }
  | PRINT expr SEMI {
        if (!suppress) printf("%.6g\n", $2);
        $$ = $2;
    }
```

• **if_cond LBRACE stmts RBRACE** - if without else: uses suppress to silence body when condition is false

• **if_cond LBRACE stmts RBRACE ELSE LBRACE stmts RBRACE** - if/else: suppresses the if-body when false, suppresses the else-body when true

```
    /* if without else */
    | if_cond LBRACE stmts RBRACE %prec LOWER_THAN_ELSE {
          if (!(int)$1) suppress--;   /* restore after body */
          $$ = $1;
      }

    /* if with else */
    | if_cond LBRACE stmts RBRACE ELSE {
          if (!(int)$1) suppress--;   /* done suppressing if-body */
          if ( (int)$1) suppress++;   /* now suppress else-body */
      }
      LBRACE stmts RBRACE {
          if ((int)$1) suppress--;    /* done suppressing else-body */
          $$ = (int)$1 ? $3 : $8;
      }
```

• **WHILE LPAREN expr RPAREN LBRACE stmts RBRACE** - checks condition and reports whether body was evaluated (single-pass)

```
    /* while loop */
    | WHILE LPAREN expr RPAREN LBRACE stmts RBRACE {
          /*
           * Body is evaluated once during single-pass reduction.
           * Full iteration requires an AST (see Limitations in report).
           */
          if (!suppress) {
              if ($3)
                  printf("[while] condition true; body evaluated (result: %.6g)\n", $6);
              else
                  printf("[while] condition false; body skipped\n");
          }
          $$ = $6;
      }
```

• **FOR LPAREN for_init SEMI expr SEMI for_update RPAREN LBRACE stmts RBRACE** - recognizes full for-loop syntax; body evaluated once

```
    /* for loop */
    | FOR LPAREN for_init SEMI expr SEMI for_update RPAREN LBRACE stmts RBRACE {
          /*
           * Grammar fully recognises for-loop syntax.
           * Body executes once during single-pass reduction.
           * Full iteration requires an AST (see Limitations in report).
           */
          if (!suppress) {
              if ($5)
                  printf("[for] condition true; body evaluated (result: %.6g)\n", $10);
              else
                  printf("[for] condition false; body skipped\n");
          }
          $$ = $10;
      }
```

• **SWITCH LPAREN expr RPAREN LBRACE case_clauses RBRACE** - parses switch with case/default clause list

```
/* switch / case / default */
| SWITCH LPAREN expr RPAREN LBRACE case_clauses RBRACE {
        if (!suppress)
            printf("[switch] on value: %.6g\n", $3);
        $$ = $3;
  }
```

Grammar Rules: Expressions
The expr non-terminal is recursive and handles all value-producing operations. Because operator precedence is declared separately, the grammar can use flat left-recursive rules without ambiguity. The semantic value $$is always a double (tagged %type <dval> expr).

Expression rules are grouped by category:

Arithmetic: The division rule checks for $3 == 0 before dividing and prints an error message if zero is detected, preventing a crash. The modulo rule casts both operands to long long before applying the % operator since C's % is defined only for integers. The power rule calls pow($1, $3) from math.h.

```
expr:
    NUMBER                              { $$ = $1; }
  | IDENT                               { $$ = get_var($1); free($1); }

  /* Arithmetic */
  | expr PLUS  expr                     { $$ = $1 + $3; }
  | expr MINUS expr                     { $$ = $1 - $3; }
  | expr MUL   expr                     { $$ = $1 * $3; }
  | expr DIV   expr                     {
        if ($3 == 0) { fprintf(stderr, "Error: division by zero\n"); $$ = 0; }
        else         { $$ = $1 / $3; }
    }
  | expr MOD   expr                     {
        if ((int)$3 == 0) { fprintf(stderr, "Error: modulo by zero\n"); $$ = 0; }
        else              { $$ = (double)((long long)$1 % (long long)$3); }
    }
  | expr POW   expr                     { $$ = pow($1, $3); }
  | MINUS expr %prec UMINUS             { $$ = -$2; }
```

Comparison: rules for ==, !=, >, >=, <, <= each evaluate to 1.0 (true) or 0.0 (false) using the ternary operator, following C convention where any non-zero value is truthy.

Logical: && and || rules apply C's short-circuit logical operators and return 1.0 or 0.0. The unary ! rule inverts the boolean value of its operand.

```
/* Comparison */
| expr EQ  expr                    { $$ = ($1 == $3) ? 1 : 0; }
| expr NEQ expr                    { $$ = ($1 != $3) ? 1 : 0; }
| expr GT  expr                    { $$ = ($1 >  $3) ? 1 : 0; }
| expr GTE expr                    { $$ = ($1 >= $3) ? 1 : 0; }
| expr LT  expr                    { $$ = ($1 <  $3) ? 1 : 0; }
| expr LTE expr                    { $$ = ($1 <= $3) ? 1 : 0; }

/* Logical */
| expr AND expr                    { $$ = ($1 && $3) ? 1 : 0; }
| expr OR  expr                    { $$ = ($1 || $3) ? 1 : 0; }
| NOT  expr                        { $$ = !$2; }

```

Trigonometric functions: sin(), cos(), and tan() each match a keyword token followed by a parenthesized expression, calling the corresponding math.h function. Arguments are in radians.

Other math functions: log() checks for non-positive arguments before calling log() since the natural logarithm is undefined for $x \leq 0$. sqrt() checks for negative arguments. abs() calls fabs() from math.h to handle floating-point absolute values.

```
/* Trigonometric */
| SIN  LPAREN expr RPAREN            { $$ = sin($3); }
| COS  LPAREN expr RPAREN            { $$ = cos($3); }
| TAN  LPAREN expr RPAREN            { $$ = tan($3); }

/* Other math */
| LOG  LPAREN expr RPAREN            {
      if ($3 <= 0) { fprintf(stderr, "Error: log of non-positive number\n"); $$ = 0; }
      else         { $$ = log($3); }
  }
| EXP  LPAREN expr RPAREN            { $$ = exp($3); }
| SQRT LPAREN expr RPAREN            {
      if ($3 < 0) { fprintf(stderr, "Error: sqrt of negative number\n"); $$ = 0; }
      else        { $$ = sqrt($3); }
  }
| ABS  LPAREN expr RPAREN            { $$ = fabs($3); }

| LPAREN expr RPAREN                 { $$ = $2; }
;
```

Error Handler: yyerror()
Bison requires a function named yyerror(const char *s) to be defined.

It is called automatically whenever the parser encounters a token sequence that does not match any grammar rule.

The implementation prints the error message to stderr.

This provides basic feedback when the user types a malformed expression, such as a missing semicolon or mismatched parenthesis, without crashing the program.

```c
void yyerror(const char *s) {
    fprintf(stderr, "Parse error: %s\n", s);
}
```

5. Integration and Compilation

Having examined the source code, this section will walk through the process of compiling and running the program.

Step 1:

Place all four project files, lexer.l, parser.y, main.c, and the Makefile, into a single folder. This folder can be named anything and placed anywhere on your machine. Keeping all files together in one directory is required for the Makefile to locate and compile them correctly.

Step 2:

Open a terminal window and use the cd command to navigate into the project folder.

For example, if the folder is named mini-compiler and is located on the Desktop, the command would be: **cd ~/Desktop/mini-compiler**

Adjust the path to match wherever you placed your folder.

```
(base) absfolder@Amines-MacBook-Pro ~ % cd Desktop
(base) absfolder@Amines-MacBook-Pro Desktop % cd "CSCE A331"
(base) absfolder@Amines-MacBook-Pro CSCE A331 % cd mini-compiler
```

Step 3:

Once inside the project directory,

Run:

```
(base) absfolder@Amines-MacBook-Pro mini-compiler % make
bison -d parser.y
bison -d parser.y
flex lexer.l
gcc -Wall -Wno-unused-function -g parser.tab.c lex.yy.c main.c -o compiler -lm
(base) absfolder@Amines-MacBook-Pro mini-compiler %
```

Step 4:

Launch the compiler.

A welcome banner will appear, and the compiler will wait for input. Enter any supported statement followed by a semicolon and press Enter to evaluate it.

Run:

```
(base) absfolder@Amines-MacBook-Pro mini-compiler % ./compiler
==========================================
   Mini Compiler  (Flex + Bison)
==========================================
Supported: arithmetic, trig, sqrt, abs,
           variables, if/else, while,
           for, switch/case, print
End each statement with ';'
Example: x = 3 * (2 + sin(1.57));
------------------------------------------

x = 3 * (2 + sin(1.57));
x = 9

y = 10;
y = 10

if (x > y) {print 1;} else {print 0;}
0
```

When finished, press Ctrl+D or Ctrl+C to exit the program.

6. Result Demonstration

We have demonstrated everything that is needed for you to understand how the program works. Now, let us go over what everything looks like on the terminal.

6.1 Arithmetic

Addition

```
5 + 3;
= 8
```

Subtraction

```
5 - 3;
= 2
```

Multiplication

```
5 * 3;
= 15
```

Division

```
15 / 5;
= 3
```

6.2 Trigonometric and Math Functions

Sin

```
sin(0);
= 0
```

Cos

```
cos(0);
= 1
```

Square Root

```
sqrt(25);
= 5
```

6.3 Variables

```
x = 10;
x = 10

y = 3;
y = 3

x + y;
= 13
```

6.4 Conditional Statements

if / else

```
if (10 >= 10) { print 10; }
10
```

while

```
while (1 > 0) { print 7; }
7
[while] condition true; body evaluated (result: 7)
```

for

```
for (i = 1; i < 5; i = i + 1) { print i; }
2
[for] condition true; body evaluated (result: 2)
```

switch

```
switch(2){ case 1: print 100; case 2: print 200; }
100
200
[switch] on value: 2
```

7. Analysis

7.1 Limitations

Several limitations arise from the choice of LALR(1) parsing without constructing an Abstract Syntax Tree (AST):

Single-pass loop evaluation. Because Bison reduces grammar rules bottom-up in a single pass, the body of a while or for loop is reduced (and thus evaluated) exactly once before the loop rule itself fires. True iteration, executing the body multiple times, requires storing the body as an AST and interpreting it repeatedly. This is the standard next step for compiler projects of this kind.

switch/case fallthrough. The parser recognizes switch/case/default syntax and reports the switch value. However, because all case bodies are reduced unconditionally during parsing, all branches execute rather than only the matching one. Proper case dispatch also requires an AST or jump-table approach.

No type system. All values are stored as C double. This means there are no integers, strings, or booleans as distinct types.

No scope or functions. Variables are global. There is no mechanism for defining or calling user-defined functions, and there is no block scoping.

No comments in source. The mini-language does not support // or /* */ comment syntax.

7.2 Possible Improvements

- AST construction: Replacing direct evaluation with AST building would enable true iterative loops, proper switch dispatch, and more sophisticated optimizations.
- User-defined functions: Adding a function declaration syntax (e.g. func name(args) { body }) and a call stack would greatly expand expressiveness.
- Type system: Introducing integer, boolean, and string types would require extending the %union and adding type-checking rules to the grammar.
- Additional math functions: floor(), ceil(), round(), pow() with two arguments, and logarithms in arbitrary bases are straightforward additions.
- Error recovery: Using Bison's error token to skip to the next semicolon on a parse error would allow the compiler to continue processing after a bad statement.
- Comment support: Adding // and /* */ patterns to the lexer is a small but useful improvement.

8. Conclusion

This project successfully demonstrates the construction of a mini compiler using Flex and Bison, covering the full development process from initial setup through implementation and analysis. Beginning with the environment configuration and toolchain requirements, the guide walks through everything needed to get the compiler built and running, including compilation instructions and example usage so the system can be tested hands-on from the start.

The supported features reflect a capable and practical subset of a real programming language. The compiler handles arithmetic and mathematical expressions with correct operator precedence, named variable management, and all required control-flow constructs including if/else, while, for, and switch/case statements. These features were not implemented in isolation; the technical breakdown of the implementation reveals how each piece connects, from regular expressions and lexical analysis in Flex, to BNF grammar design and conflict resolution in Bison, to semantic actions and the communication between lexer and parser through the yylval union.

The analysis section brings the project full circle by honestly assessing what works, what does not, and why. Limitations such as single-pass loop body evaluation are well-understood consequences of the LALR(1) parsing model rather than oversights, and the discussion of room for improvement points toward meaningful next steps, most notably AST construction and tree-walking interpretation. Taken together, these sections provide both a practical reference for building the compiler and a foundation for understanding where the work naturally leads next.

9. References

danieltan1517. (2024, February 6). *CS152 Compiler Design with Flex and Bison* [Video playlist]. YouTube. https://www.youtube.com/watch?v=LXpCVMHc1Cw&list=PL2fmKE0pL4IyKXQCFvav6up5h5lMBDWfd&index=1

Levine, J. (2009). *Flex & Bison: Text processing tools*. O'Reilly Media.